UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Laboratorio de Compiladores

Práctica 6

Parser Canónico LR(1) y Compactación LALR(1)

Jesús Eliuth Martínez Mendoza

Octubre de 2026

1. Gramática utilizada

Para construir el analizador sintáctico se utiliza la siguiente gramática aumentada:

Número	Producción
1	$S' \to S$
2	$S \to L = R$
3	$S \to R$
4	$L \to *R$
5	$L \to id$
6	$R \to L$

Los símbolos terminales son:

$$VT = \{*, id, =, \$\}$$

Los símbolos no terminales son:

$$VN = \{S, L, R\}$$

El símbolo \$ representa el fin de la entrada.

2. Conjuntos FIRST

A partir de las producciones de la gramática se obtienen los siguientes conjuntos. Como

$$L \to *R \mid id$$

se tiene

$$FIRST(L) = \{*, id\}.$$

Debido a que

$$R \to L$$

se obtiene

$$FIRST(R) = \{*, id\}.$$

Finalmente, como

$$S \to L = R \mid R$$

resulta

$$FIRST(S) = \{*, id\}.$$

Por lo tanto:

$$\boxed{FIRST(S) = FIRST(L) = FIRST(R) = \{*, id\}}$$

3. Construcción de los estados canónicos LR(1)

Se utiliza el elemento inicial:

$$[S' \to \cdot S, \$]$$

Para construir los estados se aplican las operaciones CLOSURE y GOTO, considerando los símbolos de anticipación (*lookaheads*) correspondientes.

Estado I_0

Aplicando CLOSURE al elemento inicial se obtiene:

$$I_0 = \left\{ \begin{array}{c} [S' \to \cdot S, \$], \\ [S \to \cdot L = R, \$], \\ [S \to \cdot R, \$], \\ [R \to \cdot L, \$], \\ [L \to \cdot * R, \{=, \$\}], \\ [L \to \cdot id, \{=, \$\}] \end{array} \right\}$$

Los anticipadores de las producciones de L incluyen = porque aparece después de L en la producción $S \to L = R$, y \$ porque $R \to L$ aparece al final de una producción.

Estado I_1

Se obtiene mediante $GOTO(I_0, S) = I_1$:

$$I_1 = \{[S' \to S\cdot, \$]\}$$

Este es el estado de aceptación.

Estado I_2

Se obtiene mediante $GOTO(I_0, L) = I_2$:

$$I_2 = \{[S \to L\cdot = R, \$],\ [R \to L\cdot, \$]\}$$

En este estado existe un desplazamiento con = y una reducción con \$.

Estado I_3

Se obtiene mediante $GOTO(I_0, R) = I_3$:

$$I_3 = \{[S \to R\cdot, \$]\}$$

Estado I_4

Se obtiene mediante $GOTO(I_0, *) = I_4$. Al aplicar CLOSURE se obtiene:

$$I_4 = \left\{ \begin{array}{c} [L \to * \cdot R, \{=, \$\}], \\ [R \to \cdot L, \{=, \$\}], \\ [L \to \cdot * R, \{=, \$\}], \\ [L \to \cdot id, \{=, \$\}] \end{array} \right\}$$

Estado I_5

Se obtiene mediante $GOTO(I_0, id) = I_5$:

$$I_5 = \{[L \to id\cdot, \{=, \$\}]\}$$

Estado I_6

Se obtiene mediante $GOTO(I_2, =) = I_6$. Aplicando CLOSURE:

$$I_6 = \left\{ \begin{matrix} [S \to L = \cdot R, \$], \\ [R \to \cdot L, \$], \\ [L \to \cdot * R, \$], \\ [L \to \cdot id, \$] \end{matrix} \right\}$$

Estado I_7

Se obtiene mediante $GOTO(I_4, R) = I_7$:

$$I_7 = \{[L \to *R\cdot, \{=, \$\}]\}$$

Estado I_8

Se obtiene mediante $GOTO(I_4, L) = I_8$:

$$I_8 = \{[R \to L\cdot, \{=, \$\}]\}$$

Estado I_9

Se obtiene mediante $GOTO(I_6, R) = I_9$:

$$I_9 = \{[S \to L = R\cdot, \$]\}$$

Estado I_{10}

Se obtiene mediante $GOTO(I_6, L) = I_{10}$:

$$I_{10} = \{[R \to L\cdot, \$]\}$$

Estado I_{11}

Se obtiene mediante $GOTO(I_6, *) = I_{11}$. Aplicando CLOSURE:

$$I_{11} = \left\{ \begin{matrix} [L \to * \cdot R, \$], \\ [R \to \cdot L, \$], \\ [L \to \cdot * R, \$], \\ [L \to \cdot id, \$] \end{matrix} \right\}$$

Estado I_{12}

Se obtiene mediante $GOTO(I_6, id) = I_{12}$:

$$I_{12} = \{[L \to id\cdot, \$]\}$$

Estado I_{13}

Se obtiene mediante $GOTO(I_{11}, R) = I_{13}$:

$$I_{13} = \{[L \to *R\cdot, \$]\}$$

Por lo tanto, la colección canónica LR(1) contiene:

$$\boxed{14 \text{ estados}}$$

4. Transiciones del autómata LR(1)

Las transiciones obtenidas mediante GOTO son:

Estado	Símbolo	Estado destino
I0	S	I1
I0	L	I2
I0	R	I3
I0	*	I4
I0	id	I5
I2	=	I6
I4	R	I7
I4	L	I8
I4	*	I4
I4	id	I5
I6	R	I9
I6	L	I10
I6	*	I11
I6	id	I12
I11	R	I13
I11	L	I10
I11	*	I11
I11	id	I12

5. Compactación LALR(1)

Para obtener el autómata LALR(1), se identifican los estados que tienen el mismo núcleo LR(1). Estos estados pueden fusionarse uniendo sus respectivos conjuntos de anticipación. En nuestra colección se identifican cuatro pares:

Estados originales	Estado fusionado
I4 e I11	J4
I5 e I12	J5
I7 e I13	J7
I8 e I10	J8

Los demás estados permanecen sin cambios. La correspondencia completa es:

Estado LALR(1)	Estados LR(1) originales
J0	I0
J1	I1
J2	I2
J3	I3
J4	I4, I11
J5	I5, I12
J6	I6
J7	I7, I13
J8	I8, I10
J9	I9

De esta manera, el número de estados se reduce de 14 a 10:

$$\boxed{14 \longrightarrow 10}$$

6. Estados resultantes LALR(1)

Los estados fusionados principales quedan de la siguiente manera.

Estado J_4

$$J_4 = \left\{ \begin{matrix} [L \to * \cdot R, \{=, \$\}], \\ [R \to \cdot L, \{=, \$\}], \\ [L \to \cdot * R, \{=, \$\}], \\ [L \to \cdot id, \{=, \$\}] \end{matrix} \right\}$$

Estado J_5

$$J_5 = \{[L \to id\cdot, \{=, \$\}]\}$$

Estado J_7

$$J_7 = \{[L \to *R\cdot, \{=, \$\}]\}$$

Estado J_8

$$J_8 = \{[R \to L\cdot, \{=, \$\}]\}$$

Los demás estados conservan los elementos de sus estados LR(1) originales.

7. Tabla ACTION

La tabla ACTION se construye utilizando los desplazamientos, las reducciones y la aceptación obtenidos del autómata LALR(1). Las producciones utilizadas para las reducciones son:

Reducción	Producción
r2	$S \to L = R$
r3	$S \to R$
r4	$L \to *R$
r5	$L \to id$
r6	$R \to L$

La tabla resultante es:

Estado	*	id	=	$
J0	s4	s5	–	–
J1	–	–	–	acc
J2	–	–	s6	r6
J3	–	–	–	r3
J4	s4	s5	–	–
J5	–	–	r5	r5
J6	s4	s5	–	–
J7	–	–	r4	r4
J8	–	–	r6	r6
J9	–	–	–	r2

Donde: s representa desplazamiento; r representa reducción; acc representa aceptación; – representa una acción de error.

8. Tabla GOTO

La tabla GOTO contiene las transiciones entre estados utilizando los no terminales S, L y R.

Estado	S	L	R
J0	1	2	3
J1	–	–	–
J2	–	–	–
J3	–	–	–
J4	–	8	7
J5	–	–	–
J6	–	8	9
J7	–	–	–
J8	–	–	–
J9	–	–	–

9. Resolución del conflicto SLR(1)

En el análisis SLR(1), las reducciones se determinan utilizando los conjuntos FOLLOW de los no terminales. En este caso, después de reconocer L se presenta el estado que contiene:

$$[S \to L \cdot = R] \qquad [R \to L \cdot]$$

El primer elemento indica que debe realizarse un desplazamiento cuando aparece =. Sin embargo, en SLR(1), como

$$FOLLOW(R) = \{=, \$\}$$

también se intentaría realizar la reducción

$$R \to L$$

cuando aparece =. Esto genera un conflicto desplazamiento-reducción. En LR(1), el elemento completo conserva su anticipación específica:

$$[R \to L \cdot, \$]$$

Por lo tanto, la reducción solamente se realiza cuando aparece \$, mientras que con = se realiza el desplazamiento. Así se evita el conflicto:

$$ACTION[J_2, =] = s_6, \qquad ACTION[J_2, \$] = r_6$$

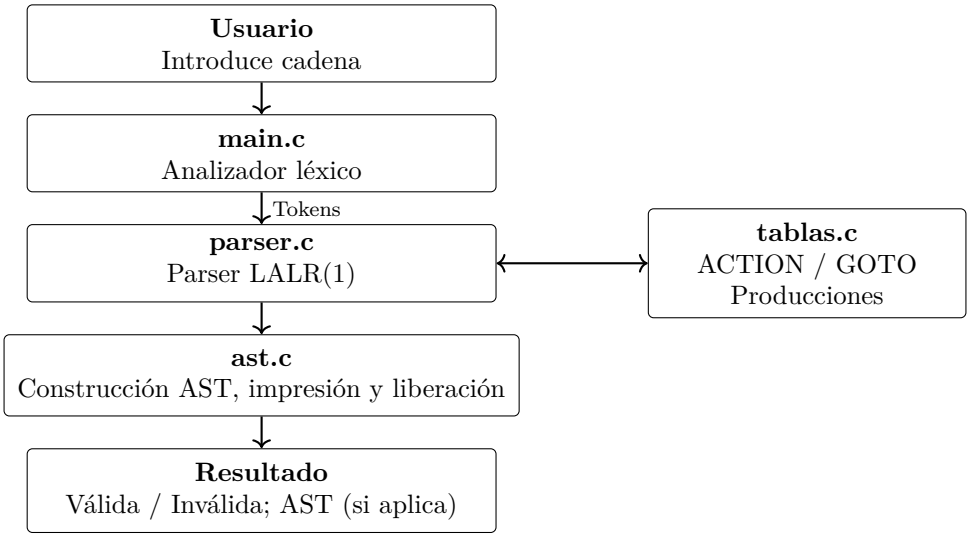
La información de anticipación permite distinguir correctamente ambas acciones.

10. Resultado de la construcción

A partir de la gramática aumentada se construyeron 14 estados canónicos LR(1). Posteriormente, se identificaron cuatro pares de estados con el mismo núcleo y se fusionaron para obtener un autómata LALR(1) de 10 estados. Las tablas ACTION y GOTO resultantes se implementaron en lenguaje C y se utilizaron para desarrollar el analizador sintáctico. La implementación permite reconocer cadenas válidas, rechazar cadenas inválidas y construir un árbol de sintaxis abstracta para las cadenas aceptadas.

11. Arquitectura del programa

El programa se organiza en cuatro componentes principales que trabajan en conjunto para realizar el análisis sintáctico LALR(1).



12. Pruebas del analizador sintáctico

Se realizaron diez pruebas para comprobar el funcionamiento del analizador: cinco cadenas válidas y cinco inválidas.

Cadena	Tokens	Resultado	Estado final	Justificación
`id`	id	Válida	I1	Se reconoce mediante $L \to id$ y $S \to R$.
`id=id`	id, =, id	Válida	I1	Cumple la producción $S \to L = R$.
`*id`	*, id	Válida	I1	Cumple $L \to *R$ y $R \to L$.
`*id=id`	*, id, =, id	Válida	I1	Cumple $S \to L = R$, con $L \to *R$.
`id=*id`	id, =, *, id	Válida	I1	Cumple $S \to L = R$, con $R \to L$ y $L \to *R$.
`id=`	id, =	Inválida	I6	Falta el operando derecho después de =.
`=id`	=, id	Inválida	I0	La gramática no permite comenzar con =.

Cadena	Tokens	Resultado	Estado final	Justificación
`id==id`	id, =, =, id	Inválida	I6	Después de =, se esperaba el inicio de R.
`**`	*, *	Inválida	I4	Después del segundo ∗, falta un identificador.
`id=id=id`	id, =, id, =, id	Inválida	I9	La gramática no permite encadenar asignaciones.

Los estados finales corresponden a los resultados obtenidos durante la ejecución de las pruebas automatizadas. Las cinco cadenas válidas fueron aceptadas y las cinco cadenas inválidas fueron rechazadas correctamente.

13. Repositorio del proyecto

`https://github.com/eliuth1805/practica6-Compiladores`